

4 Linear Operators, Images, and Convolution

Chapter Abstract. Images have local structure: nearby pixels form strokes, edges, corners, and textures. This chapter develops the mathematics that lets convolutional neural networks use that structure. A handwritten-digit example is used to define image tensors, local patches, kernels, cross-correlation, convolution, padding, stride, dilation, channels, filter banks, pooling, and translation equivariance. The central message is practical and mathematical: before a nonlinearity is applied, a convolutional layer is a structured linear operator with repeated local weights.

4.1 Running Example: Pixels, Neighborhoods, and Strokes

Chapters 1 and 3 already turned handwritten digits into arrays. This chapter uses the same kind of grayscale digit only to study the extra structure exposed by its spatial axes. Write one image as

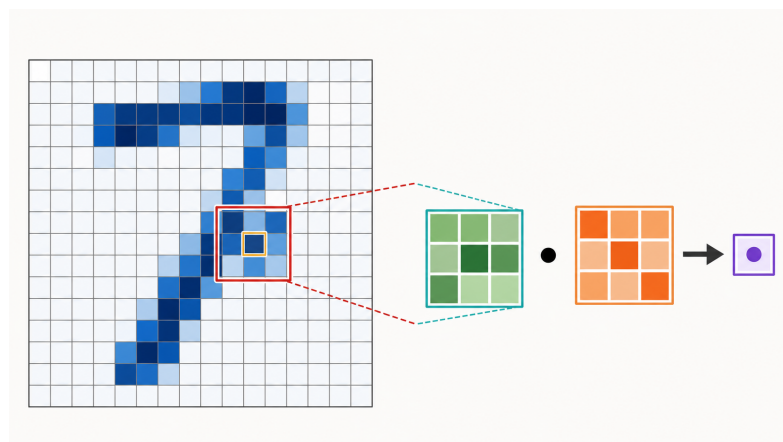
$$X \in [0, 1]^{28 \times 28},$$

where the row and column indices identify neighboring pixel locations. The new question is not how images become numbers, but how a linear operator can use neighbor relations among those numbers.

A useful mental picture is a small stencil moving over the grid. The stencil asks the same local question at many locations, which is different from giving every pixel coordinate its own unrelated rule.

Flattening keeps the values. Spatial axes keep the neighbor relation visible.

Figure 4.1: A digit image is a grid of pixel intensities. Zooming in makes the local neighborhood structure visible, and a small patch can be treated as the basic unit that a convolutional filter compares against.



Definition 4.1.1 (Spatial Image Tensor) *A spatial image tensor is an image array whose height and width axes are kept explicit so that local patches can be indexed. A grayscale image has shape $h \times w$ or $h \times w \times 1$; a color image has shape $h \times w \times c$, where c is the number of channels.*

Flattening X into $\mathbb{R}^{784}$ is useful for dense linear algebra, but the vector shape no longer identifies adjacent pixels. A stroke in a digit is not usually a single pixel; it is a local pattern across several nearby pixels. Convolutional layers make this local view explicit, which is one reason they became central in image recognition systems such as LeNet [125] and later deep-learning vision models [124].

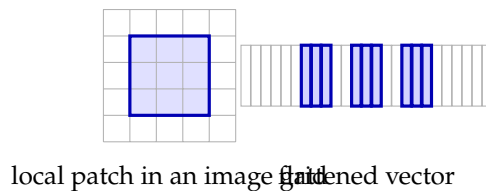


Figure 4.2: A spatial grid makes a local patch explicit. After flattening, the same values remain, but the vector shape no longer marks the neighborhood.

Pause and predict

Suppose a filter detects a bright vertical stroke at one image location. If the same stroke moves two pixels to the right, predict which output location should respond and whether the detector should need new position-specific weights.

The practical question for the rest of the chapter is this: how can a model look for the same small pattern at many different locations without learning a separate unrelated parameter for every pixel position?

4.2 Images as Tensors

Chapter 3 adopted a channel-last convention for image tensors while noting that some libraries use channel-first order. This chapter continues with channel-last notation: a single image is written as $h \times w \times c$. Keeping h and w adjacent makes the patch and kernel formulas easier to read.

Definition 4.2.1 (Image Layout Convention) *For this chapter, a single image has shape $h \times w \times c$, where h is height, w is width, and c is the number of channels. A batch of m images has shape $m \times h \times w \times c$.*

The channel axis stores measurements at one location. It is not another batch axis.

Object	Shape in this chapter	Meaning
One grayscale digit	$28 \times 28 \times 1$	Height, width, one intensity channel
Batch of grayscale digits	$m \times 28 \times 28 \times 1$	Example axis plus image axes
One RGB image	$h \times w \times 3$	Red, green, and blue channels
Batch of RGB images	$m \times h \times w \times 3$	Batch axis plus color image axes

Definition 4.2.2 (Channel Axis) *Using the axis language from Chapter 3, the channel axis stores different measurements at the same spatial location. In an RGB image, $X_{i,j,1}$, $X_{i,j,2}$, and $X_{i,j,3}$ are the three color measurements at row i and column j . In a learned feature map, channels store learned feature responses at the same spatial location.*

The channel axis is not a second batch axis. If an input has shape $m \times h \times w \times c$, the index $r \in \{1, \dots, m\}$ chooses an example, while the channel index chooses a measurement inside that example. Mixing these axes changes the mathematical object being fed to a model.

Example 4.2.1 (A Digit Batch) A batch of 32 grayscale 28×28 images has shape

$$32 \times 28 \times 28 \times 1.$$

The entry $X_{7,12,20,1}$ is the intensity in example 7, row 12, column 20, channel 1. A convolutional layer will usually operate on the last three axes for each example, while sharing the same learned weights across the batch.

Pause and predict

For a batch of 16 RGB images of size 64×64 , predict the channel-last shape. Then decide which axis changes when the same image is converted from grayscale to RGB.

The remaining formulas assume this layout. Data stored as $m \times c \times h \times w$ must be permuted before those formulas are used; otherwise a kernel would slide across a channel or batch dimension instead of the height and width axes.

4.3 Dense Linear Maps on Flattened Images

Use the previously introduced flattening operation as a baseline. For a 28×28 grayscale digit,

$$x = \text{vec}(X) \in \mathbb{R}^{784}.$$

In this baseline, a dense affine layer computes the output vector from the flattened image as

$$z = Wx + b,$$

where $W \in \mathbb{R}^{q \times 784}$, $b \in \mathbb{R}^q$, and $z \in \mathbb{R}^q$.

Definition 4.3.1 (Row-Wise Image Vectorization Convention) *In this chapter, $\text{vec}(X)$ lists the entries of a spatial image tensor row by row. For a grayscale $h \times w$ image, $\text{vec}(X) \in \mathbb{R}^{hw}$.*

A dense layer can connect every pixel to every output, but it does not encode which pixels were neighbors.

Flattening is not wrong; it is often useful near the end of a classifier. The issue is that a dense layer has no built-in preference for nearby pixels. The weight connecting pixel 12, 13 to a hidden unit is unrelated to the weight connecting pixel 12, 14 unless training discovers a relation from data.

Example 4.3.1 (Parameter Count After Flattening) Mapping a flattened 28×28 image to 64 hidden features requires

$$64 \cdot 784 + 64 = 50240$$

trainable parameters. Mapping the same image directly to ten class scores requires

$$10 \cdot 784 + 10 = 7850$$

parameters. These counts are manageable for small digits, but they grow quickly for larger images.

Definition 4.3.2 (Receptive Field) *For one output coordinate of a layer, the receptive field is the set of input coordinates that can affect that output coordinate. In a dense layer applied to a flattened image, every output coordinate can depend on every input pixel.*

The dense receptive field is mathematically general. It is also a weak match to the way many image patterns behave. A small vertical stroke can appear near the left side, center, or right side of a digit. If each location has unrelated dense weights, the model must learn the same kind of local detector many times. Convolution changes the linear map so that small local weights are reused across locations.

4.4 Local Patches and Sliding Dot Products

A convolutional layer begins with a local patch. Instead of connecting an output to every pixel, it looks at a small window, such as 3×3 , computes a dot product with a small array of weights, and then slides the same weights to another location.

One kernel defines one local detector. Sliding applies that detector at many spatial locations.

Definition 4.4.1 (Local Patch and Kernel) For an image $X \in \mathbb{R}^{h \times w}$ and window size $k_h \times k_w$, the local patch at valid location (i, j) is the subarray

$$P_{i,j} = (X_{i+u,j+v})_{0 \leq u < k_h, 0 \leq v < k_w} \in \mathbb{R}^{k_h \times k_w}.$$

A kernel, or filter, is a weight array $K \in \mathbb{R}^{k_h \times k_w}$ with the same spatial shape as the patch it is compared against.

For a one-channel image, a $k_h \times k_w$ kernel K produces one number from a $k_h \times k_w$ patch by

$$\langle K, \text{patch} \rangle = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v} \text{patch}_{u,v}.$$

The result is a feature response. Sliding the kernel over the image produces a grid of feature responses called a spatial feature map.

A sliding dot-product response with a $k_h \times k_w$ kernel is local. The output at location (i, j) depends only on the input entries

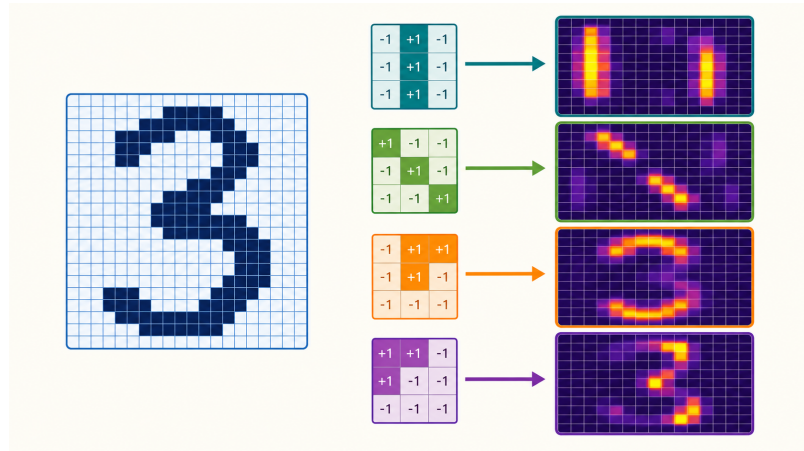
$$X_{i+u,j+v}, \quad 0 \leq u < k_h, \quad 0 \leq v < k_w.$$

The response is a sum of terms

$$K_{u,v} X_{i+u,j+v}.$$

Only entries with indices of the form $(i+u, j+v)$ appear in the sum. An input entry outside that index set has coefficient zero in this output coordinate.

Figure 4.3: Different filters can respond strongly to different local stroke patterns. Sliding each detector across the same input produces a corresponding feature map.



Example 4.4.1 (A Hand Calculation) Let

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}.$$

At the upper-left patch,

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix},$$

the response is

$$1 \cdot 1 + (-1) \cdot 2 + 0 \cdot 0 + 2 \cdot 1 = 1.$$

Sliding the same kernel over all valid 2×2 patches gives

$$\begin{bmatrix} 1 & 8 \\ 1 & -2 \end{bmatrix}.$$

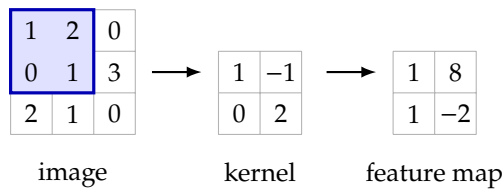


Figure 4.4: A small kernel is dotted with one local patch, then reused at other valid patch locations to form a spatial feature map.

Pause and predict

In the example above, which output entry would change if only the lower-right input pixel changed? Which output entries would be unaffected?

The important modeling idea is not that the shown kernel is special. In a neural network, these kernel entries are trainable parameters. During training, a filter can become a detector for a local stroke, edge, corner, or texture when such a detector helps reduce the loss.

4.5 Cross-Correlation and Convolution

The sliding dot product used by many deep-learning libraries is technically cross-correlation. Deep-learning texts often still call the resulting layer a convolutional layer, and this convention is standard enough that both names must be understood [67].

Most neural-network libraries use correlation arithmetic while keeping the architecture name “convolution.”

Definition 4.5.1 (Valid Cross-Correlation) Let $X \in \mathbb{R}^{h \times w}$ and $K \in \mathbb{R}^{k_h \times k_w}$, with $k_h \leq h$ and $k_w \leq w$. The valid cross-correlation of X by K is the array $Y = \text{corr}(X, K) \in \mathbb{R}^{(h-k_h+1) \times (w-k_w+1)}$ with

$$Y_{i,j} = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v} X_{i+u,j+v},$$

for every valid output location (i, j) .

Mathematical convolution flips the kernel before sliding it. In two dimensions, the valid discrete convolution can be written as

$$(\text{conv}(X, K))_{i,j} = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v} X_{i+k_h-1-u, j+k_w-1-v}.$$

Convolution can be written as correlation with a rotated kernel. Let $\tilde{K}$ be the 180° -rotated kernel

$$\tilde{K}_{u,v} = K_{k_h-1-u, k_w-1-v}.$$

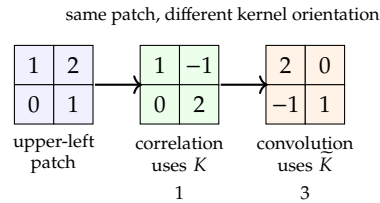
At each valid output location,

$$\begin{aligned} \text{corr}(X, \tilde{K})_{i,j} &= \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \tilde{K}_{u,v} X_{i+u,j+v} \\ &= \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{k_h-1-u, k_w-1-v} X_{i+u,j+v}. \end{aligned}$$

Renaming $a = k_h - 1 - u$ and $b = k_w - 1 - v$ gives the convolution formula, so

$$\text{conv}(X, K) = \text{corr}(X, \tilde{K}).$$

Figure 4.5: Cross-correlation dots the patch with the learned kernel as stored. Mathematical convolution first rotates the kernel by 180° , so the same patch can produce a different hand-computed response.



Remark 4.5.1 (Deep-Learning Convolution Convention) In this chapter, a convolutional layer uses the cross-correlation formula unless a kernel flip is stated explicitly. This matches the operation commonly implemented in deep-learning libraries. The layer is still called convolutional because that is the standard architecture name.

Example 4.5.1 (The Kernel Flip) For

$$K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix},$$

the flipped kernel used by true convolution is

$$\begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}.$$

The upper-left valid convolution response for the example image in Section 4.4 is

$$2 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 1 \cdot 1 = 3,$$

while the cross-correlation response was 1.

When a kernel is learned, the flip does not usually change the family of functions available to the model. A learned kernel can learn either orientation. The convention still matters when checking a hand calculation against software output or when using a fixed filter from signal processing.

Pause and predict

If a hand calculation gives 3 for a fixed edge filter but a library reports 1 on the same patch, name the convention you would check before calling it a bug. Then predict which result would change if the stored kernel were rotated before calling the library.

4.6 Convolution as a Linear Operator

For a fixed kernel, cross-correlation is linear in the input image. This fact is more than a formal observation. It means a convolutional layer before its activation is a linear map with a special repeated-weight pattern.

Convolution is a constrained linear map: sparse local connections plus repeated weights.

Definition 4.6.1 (Convolution Matrix) For a fixed one-dimensional kernel $k = (k_1, \dots, k_r)$ and input length $n \geq r$, the valid cross-correlation matrix is $C_k \in \mathbb{R}^{(n-r+1) \times n}$ with entries

$$(C_k)_{i,j} = \begin{cases} k_{j-i+1}, & 1 \leq j - i + 1 \leq r, \\ 0, & \text{otherwise.} \end{cases}$$

For $x \in \mathbb{R}^n$, the product $C_k x$ is the valid sliding dot-product output. For $n = 5$ and $k = (a, b, c)$, this matrix is

$$C_k = \begin{bmatrix} a & b & c & 0 & 0 \\ 0 & a & b & c & 0 \\ 0 & 0 & a & b & c \end{bmatrix}.$$

Indeed,

$$C_k x = \begin{bmatrix} ax_1 + bx_2 + cx_3 \\ ax_2 + bx_3 + cx_4 \\ ax_3 + bx_4 + cx_5 \end{bmatrix}.$$

The same three kernel entries appear in each row, shifted one position to the right. A dense matrix with the same input and output sizes would have $3 \cdot 5 = 15$ independent weights. This structured matrix has only 3. Such a shifted-diagonal pattern is called Toeplitz: a matrix is Toeplitz when its entries are constant along each diagonal $j - i = \text{constant}$. A block-Toeplitz matrix has the same constant-diagonal pattern after scalar entries are replaced by matrix blocks.

For a fixed kernel K , the map $X \mapsto \text{corr}(X, K)$ is linear in X . For images X and Z with the same shape and scalars α, β ,

$$\text{corr}(\alpha X + \beta Z, K) = \alpha \text{corr}(X, K) + \beta \text{corr}(Z, K).$$

At each valid output location,

$$\begin{aligned} \text{corr}(\alpha X + \beta Z, K)_{i,j} &= \sum_{u,v} K_{u,v} (\alpha X_{i+u,j+v} + \beta Z_{i+u,j+v}) \\ &= \alpha \sum_{u,v} K_{u,v} X_{i+u,j+v} + \beta \sum_{u,v} K_{u,v} Z_{i+u,j+v}. \end{aligned}$$

The final line is the corresponding entry of $\alpha \text{corr}(X, K) + \beta \text{corr}(Z, K)$.

Theorem 4.6.1 (Structured Matrix Representation of Valid Correlation)

Fix $K \in \mathbb{R}^{k_h \times k_w}$ and flatten images row by row. There is a matrix

$$C_K \in \mathbb{R}^{(h-k_h+1)(w-k_w+1) \times hw}$$

such that

$$\text{vec}(\text{corr}(X, K)) = C_K \text{vec}(X)$$

for every $X \in \mathbb{R}^{h \times w}$. Each row of C_K has nonzero entries only at the coordinates of one local patch, and those nonzero entries are the kernel weights. In one dimension this matrix is Toeplitz; in two dimensions it is a sparse block-Toeplitz matrix with Toeplitz blocks.

Proof. Create one row of C_K for each valid output location (i, j) . In that row, place coefficient $K_{u,v}$ in the column corresponding to input coordinate $(i + u, j + v)$, for all $0 \leq u < k_h$ and $0 \leq v < k_w$, and place zeros elsewhere. Multiplying this row by $\text{vec}(X)$ produces exactly

$$\sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v} X_{i+u,j+v},$$

the valid correlation response at (i, j) . Stacking these rows gives all output locations. $\square$

The two-dimensional case is the same linear-algebra idea with more indexing. Before a nonlinearity is applied, valid correlation is multiplication

by a sparse matrix. Each row reads one local patch, and the repeated coefficients in nearby rows are the shared kernel weights.

$$\begin{bmatrix} a & b & c & 0 & 0 \\ 0 & a & b & c & 0 \\ 0 & 0 & a & b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} ax_1 + bx_2 + cx_3 \\ ax_2 + bx_3 + cx_4 \\ ax_3 + bx_4 + cx_5 \end{bmatrix}.$$

Figure 4.6: A one-dimensional valid cross-correlation can be written as matrix multiplication by a sparse structured matrix.

This matrix view gives the convolution-specific version of the activation discussion from Chapter 2. With a fixed kernel, the convolution is the linear map represented by the shifted-weight matrix. A following ReLU keeps positive responses and zeros the rest, so a convolution–ReLU block is piecewise linear over image tensors rather than a new kind of linear operator.

4.7 Padding, Stride, Dilation, and Output Shapes

Convolutional layers are composed many times in image models. The shapes must match from one layer to the next. Padding, stride, and dilation are the main controls for the spatial output shape. Detailed convolution arithmetic is often presented as a practical topic because mistakes in these quantities change the tensors passed to later layers [56].

Stride changes how many patch positions are sampled. Padding changes which values boundary patches include.

Definition 4.7.1 (Convolutional Shape Parameters) *For an input with height h and width w , kernel size $k_h \times k_w$, padding p_h, p_w , stride s_h, s_w , and dilation d_h, d_w , the effective kernel sizes are*

$$k_{h,\text{eff}} = d_h(k_h - 1) + 1, \quad k_{w,\text{eff}} = d_w(k_w - 1) + 1.$$

The output height and width count the valid sampled positions of this effective kernel after padding.

Proposition 4.7.1 (Convolutional Output Shape) *For the shape parameters above, the output height and width are*

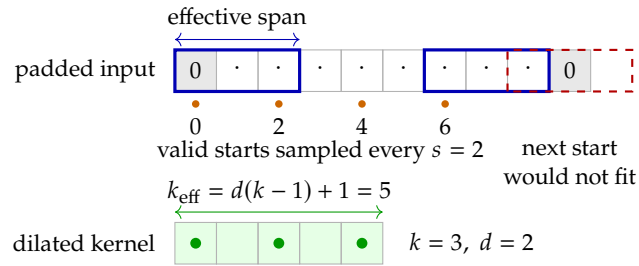
$$h_{\text{out}} = \left\lfloor \frac{h + 2p_h - d_h(k_h - 1) - 1}{s_h} + 1 \right\rfloor, \\ w_{\text{out}} = \left\lfloor \frac{w + 2p_w - d_w(k_w - 1) - 1}{s_w} + 1 \right\rfloor.$$

Proof. Along the height axis, padding creates an effective input length $h + 2p_h$. The last sampled kernel position must fit an effective span of $d_h(k_h - 1) + 1$. Starting positions are sampled every s_h entries, so the number of valid starts is

$$\left\lfloor \frac{h + 2p_h - (d_h(k_h - 1) + 1)}{s_h} \right\rfloor + 1.$$

This is the stated height formula. The width formula is identical with the width parameters. $\square$

Figure 4.7: Counting valid kernel starts. Padding extends the input, stride samples starts, and dilation enlarges the effective span.



Padding adds values around the boundary of the image before the kernel is applied. Zero padding is common. Stride controls how far the sliding window moves between neighboring outputs. Dilation spaces out the entries of the kernel, so a 3×3 kernel with dilation 2 covers the span of a 5×5 region while still using only nine learned spatial weights.

Input and kernel	Padding	Stride	Output shape
$28 \times 28, 3 \times 3$	0	1	26×26
$28 \times 28, 3 \times 3$	1	1	28×28
$28 \times 28, 3 \times 3$	1	2	14×14
$28 \times 28, 5 \times 5$	0	1	24×24

Example 4.7.1 (Shape Check for the Running Digit) A 3×3 kernel with padding 1, stride 1, and dilation 1 applied to a 28×28 digit gives

$$h_{\text{out}} = \left\lfloor \frac{28 + 2 - 1(3 - 1) - 1}{1} + 1 \right\rfloor = 28.$$

This layer preserves spatial height and width. With stride 2, the same calculation gives 14, so the feature map is downsampled.

Pause and predict

Before using the formula, decide whether increasing stride should make the output larger, smaller, or unchanged. Then check the formula with $h = 28$, $k_h = 3$, $p_h = 1$, and $s_h = 2$.

In the table above, stride 2 changes the 28×28 grid into 14×14 . The next layer receives fewer patch positions, and any skip connection or flattening step after it has to use that smaller grid.

4.8 Channels, Filter Banks, and Feature Maps

Real convolutional layers usually have many input channels and many output channels. A grayscale input has one channel, an RGB input has

three channels, and a hidden layer may have dozens or hundreds of learned feature channels.

Each output channel has its own filter, and each filter spans all input channels.

Definition 4.8.1 (Multi-Channel Filter Bank) For an input tensor $X \in \mathbb{R}^{h \times w \times c_{\text{in}}}$, a convolutional layer with c_{out} output channels uses a weight tensor

$$W \in \mathbb{R}^{k_h \times k_w \times c_{\text{in}} \times c_{\text{out}}}$$

and a bias vector $b \in \mathbb{R}^{c_{\text{out}}}$. Each output channel is produced by one filter that combines all input channels.

For output channel r , the valid cross-correlation formula is

$$Y_{i,j,r} = b_r + \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \sum_{c=1}^{c_{\text{in}}} W_{u,v,c,r} X_{i+u,j+v,c}.$$

The sums over u and v move through a local spatial patch. The sum over c combines measurements or features at the same patch location across all input channels.

Example 4.8.1 (Parameter Count for a Multi-Channel Filter Bank) The parameter count is

$$k_h k_w c_{\text{in}} c_{\text{out}} + c_{\text{out}},$$

including one bias per output channel. For an RGB input with $c_{\text{in}} = 3$, a 5×5 filter bank with $c_{\text{out}} = 12$ has

$$5 \cdot 5 \cdot 3 \cdot 12 + 12 = 912$$

parameters. If the same $16 \times 16 \times 3$ image were flattened and mapped to 12 features by a dense layer, the count would be

$$(16 \cdot 16 \cdot 3) \cdot 12 + 12 = 9228.$$

The convolutional layer is smaller because the same $5 \times 5 \times 3$ filter is reused at every spatial location, not because the channels disappear.

Example 4.8.2 (Multi-channel patches as an im2col matrix) For a $3 \times 3 \times 2$ input and a 2×2 spatial kernel, each valid output location uses $2 \cdot 2 \cdot 2 = 8$ input numbers. One patch vector is

$$p_{0,0} = (X_{0,0,1}, X_{0,0,2}, X_{0,1,1}, X_{0,1,2}, X_{1,0,1}, X_{1,0,2}, X_{1,1,1}, X_{1,1,2}).$$

Stacking the four valid patch vectors gives a matrix

$$P_X \in \mathbb{R}^{4 \times 8}.$$

If there are c_{out} filters, flatten each filter into one column of

$$W_{\text{col}} \in \mathbb{R}^{8 \times c_{\text{out}}}.$$

Then the valid multi-channel convolution can be written as

$$Y_{\text{col}} = P_X W_{\text{col}} + \mathbf{1}b^T,$$

where $Y_{\text{col}} \in \mathbb{R}^{4 \times c_{\text{out}}}$ lists the output locations as rows. This is the same structured linear-operator idea as the one-channel matrix view, with each row now reading a multi-channel local patch.

The output Y is a stack of spatial feature maps. Chapter 2 used “feature map” for a representation function ϕ ; here the phrase means one output grid $Y_{:,r}$ produced by one filter in the bank. Early filters in an image model often learn simple local patterns, and later filters combine earlier feature-map grids into more abstract patterns [17].

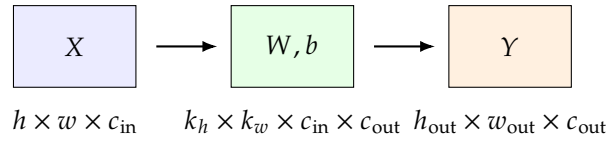


Figure 4.8: A filter bank maps an input tensor to a stack of output feature maps.

The output-channel index r names a separate learned detector. Adding more detectors increases the kernel tensor by $k_h k_w c_{\text{in}}$ weights at a time, so the same patch calculation scales from one filter to a filter bank.

4.9 Parameter Sharing, Locality, and Translation Equivariance

Convolution is useful for images because of three related ideas: locality, parameter sharing, and translation equivariance. Locality says a small output feature only sees a local input patch. Parameter sharing says the same kernel weights are reused at many spatial locations. Translation equivariance says that, away from boundary effects, shifting the input shifts the feature map in the same way.

Equivariance means the response moves with the input. It does not mean the response is unchanged.

Definition 4.9.1 (Local Shared-Weight Structure) *A convolutional layer has local shared weights when its output can be written, away from boundary conventions, as*

$$Y_{i,j,o} = b_o + \sum_{u,v,c} W_{u,v,c,o} X_{i+u,j+v,c}.$$

Locality is the restriction to offsets (u, v) in a fixed finite window. Sharing is the use of the same weights $W_{u,v,c,o}$ for every spatial location (i, j) .

The sparse-matrix view makes sharing visible. In the matrix

$$\begin{bmatrix} a & b & c & 0 & 0 \\ 0 & a & b & c & 0 \\ 0 & 0 & a & b & c \end{bmatrix},$$

the entries a, b, c are reused in shifted positions. A dense matrix of the same size would usually contain unrelated entries in every row.

Remark 4.9.1 (Structured linear algebra behind equivariance) The repeated shifted rows are not only a parameter-saving trick. They are the linear-algebra reason that the same detector is applied at neighboring locations. If the input shifts by one interior position, the local patch read by row r after the shift is the local patch that row $r + 1$ read before the shift. Because the rows reuse the same coefficients, the response shifts with the input. Untied dense rows would generally break this relationship.

Definition 4.9.2 (Finite-Grid Translation Equivariance) *Let T_δ be an operator that shifts an image by displacement δ . A map F is translation equivariant on a set of locations if*

$$F(T_\delta X) = T_\delta F(X)$$

on those locations. For finite images, boundary locations are excluded unless the boundary convention is included in the definition of T_δ and F .

Theorem 4.9.1 (Interior Equivariance of Valid Correlation) *Fix a kernel K . Let T_δ denote the shift*

$$(T_\delta X)_{a,b} = X_{a+\delta_1, b+\delta_2}$$

on indices for which the right-hand side is defined. At every interior output location where the shifted input patch and shifted output entry are both defined,

$$\text{corr}(T_\delta X, K)_{i,j} = (T_\delta \text{corr}(X, K))_{i,j}.$$

Proof. At such an interior output location,

$$\begin{aligned}
 \text{corr}(T_\delta X, K)_{i,j} &= \sum_{u,v} K_{u,v} (T_\delta X)_{i+u,j+v} \\
 &= \sum_{u,v} K_{u,v} X_{i+\delta_1+u,j+\delta_2+v} \\
 &= \text{corr}(X, K)_{i+\delta_1,j+\delta_2} \\
 &= (T_\delta \text{corr}(X, K))_{i,j}.
 \end{aligned}$$

The equalities are restricted to interior locations so that every referenced input patch and output entry exists. $\square$

Thus shared local weights make shifted interior patches see the same coefficients, so the response shifts with the input. This interpretation motivates the converse question: under a clean translation action, does linear equivariance force a shared convolution-like structure?

Theorem 4.9.2 (Linear equivariant maps are convolutional, informal)
On an infinite grid, or on a finite grid with a circular boundary convention, every linear map that commutes with all translations has convolutional form: its output at each location is obtained by applying the same coefficients to a translated copy of the input. If the map is also local, those coefficients have finite spatial support.

The theorem reverses the direction of the previous calculation. Valid correlation shows that shared local weights imply translation equivariance. The representation result says that, under linearity and a clean translation action, translation equivariance forces a shared convolution-like structure. The same principle generalizes from translations to other symmetry groups in group-convolution constructions [40].

Example 4.9.1 (Why the Boundary Caveat Matters) If a bright stroke is shifted one pixel to the right in the middle of a large image, the same vertical-stroke filter gives the same response pattern shifted one pixel to the right. If the stroke is already at the right boundary, padding or cropping changes which pixels are available to the filter. Exact equivariance can fail at that boundary.

Pause and predict

If every row of the sparse matrix used unrelated entries instead of the same a, b, c , would the map still express the same detector sliding across locations? Predict what would happen to translation equivariance.

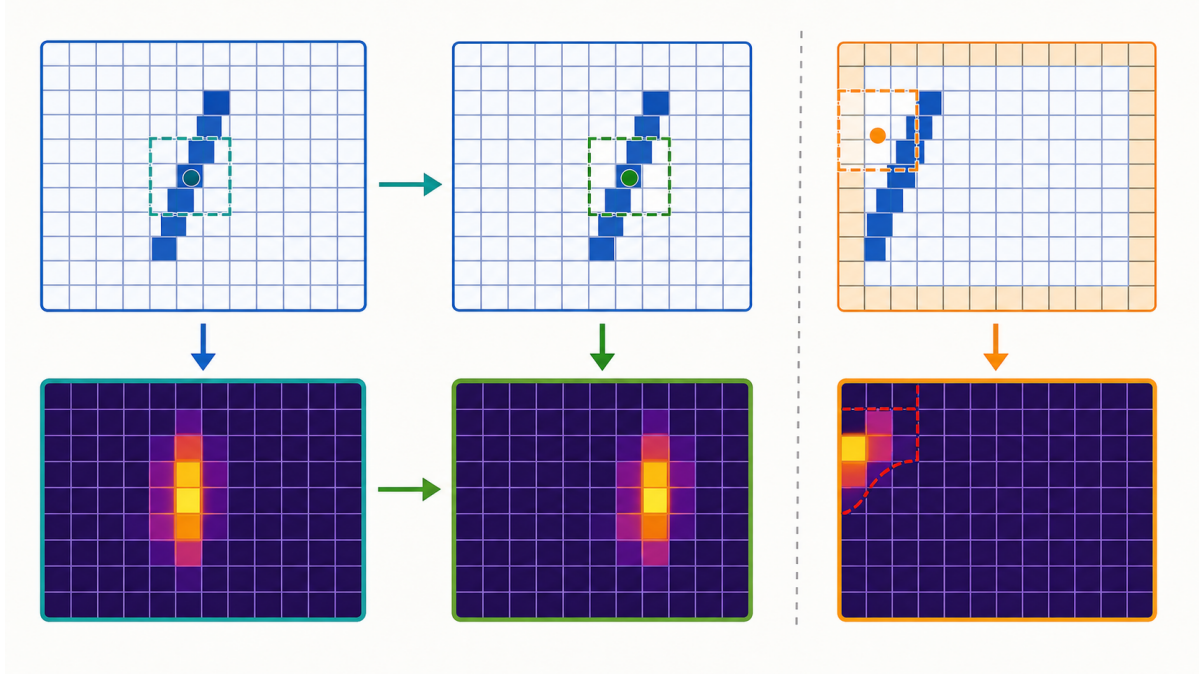


Figure 4.9: Away from the boundary, shifting the input stroke shifts the feature response in the same direction. At the boundary, the available local patch changes, so the response can be clipped or distorted.

Remark 4.9.2 (Padding and stride change the exact statement) Padding and stride qualify equivariance. Circular padding can make finite-grid shifts exact; zero padding depends on the exterior convention. Stride- S outputs remain equivariant only to shifts aligned with the sampled grid. The shared-weight calculation is still local and translation-aware, but the finite-grid operator has to state its boundary and sampling convention.

Convolution encodes position sharing, so it handles translations more directly than an unconstrained dense map on flattened pixels. Rotations, scale changes, and boundary artifacts require additional architectural or data choices.

4.10 Pooling, Nonlinearities, and a Minimal CNN Classifier

With the convolutional pieces in place, a minimal digit classifier can be read as a shape pipeline:

image $\rightarrow$ convolution $\rightarrow$ ReLU
 $\rightarrow$ pooling $\rightarrow$ flatten $\rightarrow$ linear classifier $\rightarrow$ softmax .

The new bookkeeping in this pipeline is where spatial resolution changes and where the feature-map stack is flattened for the final classifier.

Definition 4.10.1 (Local Pooling Operator) Let $X \in \mathbb{R}^{H \times W \times C}$, let $\mathcal{W}_{i,j}$ be the spatial window assigned to output location (i, j) , and let g be an aggregation function. A channelwise local pooling operator has entries

$$P_{i,j,c} = g(\{X_{u,v,c} : (u, v) \in \mathcal{W}_{i,j}\}).$$

Max pooling uses $g = \max$; average pooling uses the arithmetic mean.

Pooling is not a trainable convolutional filter. It is a fixed local aggregation. Max pooling keeps the strongest local response, while average pooling smooths local responses. Both reduce spatial resolution when used with stride greater than one.

Pooling trades spatial detail for smaller feature maps while keeping channels separate.

Example 4.10.1 (A Minimal Shape Pipeline) Consider a $28 \times 28 \times 1$ digit. A 3×3 convolution with padding 1, stride 1, and 8 output channels gives

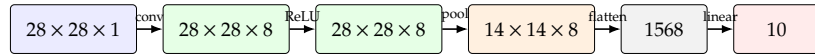
$$28 \times 28 \times 8.$$

Applying ReLU does not change the shape. A 2×2 pooling layer with stride 2 gives

$$14 \times 14 \times 8.$$

Flattening gives a vector of length $14 \cdot 14 \cdot 8 = 1568$, which can be sent to a dense layer producing ten class scores.

Figure 4.10: A minimal convolutional classifier can be understood by tracking tensor shapes through each operation.



The first convolution in this example has

$$3 \cdot 3 \cdot 1 \cdot 8 + 8 = 80$$

parameters. The final dense classifier has

$$1568 \cdot 10 + 10 = 15690$$

parameters. The contrast is the point: the convolution spends few parameters on reusable local detectors, while the dense layer mixes the extracted feature grid into ten class scores.

Chapter 5 supplies the backward pass for this pipeline, including the way shared convolutional weights accumulate gradient contributions from many spatial locations.

Pause and predict

If the pooling layer were removed from the example pipeline, what would be the length of the flattened vector before the final classifier? Predict whether the dense classifier would have more or fewer parameters, and what spatial tradeoff the pooling layer was making.

4.11 Summary

Images should not be flattened too early. Height, width, and channel axes tell the model which numbers are neighbors, which numbers are measurements at the same location, and which numbers belong to different examples.

Valid cross-correlation with a fixed kernel is a sparse linear map. A $3 \times 3 \times c_{\text{in}} \times c_{\text{out}}$ filter bank repeats the same local weights across the grid, which is exactly the parameter sharing used by convolutional layers.

The routine checks are concrete: compute the output shape after padding, stride, and dilation; count the channels in each filter bank; and remember that equivariance is cleanest away from image boundaries. After pooling and nonlinearities, these structured local responses become the feature vector that the classifier reads.

The payoff is practical. These ledgers say whether the model is looking for a local pattern wherever it appears, or merely learning a long list of coordinate-specific weights.

4.12 Exercises

Use the following ten prompts to check shape arithmetic, local filtering, and the sparse-matrix view of convolution. Show the calculation whenever a numerical answer is requested.

- Exercise 1.** State the shape of a batch of 64 RGB images with height 32 and width 32 using the channel-last convention from this chapter. Identify the batch, height, width, and channel axes.
- Exercise 2.** Compare two ways to produce 20 features from a 28×28 grayscale image: a dense layer on the flattened image and a 3×3 convolution with 20 output channels and one bias per channel. Compute both parameter counts and explain which count uses spatial weight sharing.
- Exercise 3.** Let $X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}$ and $K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$. Compute the valid cross-correlation output at the upper-left patch.
- Exercise 4.** Using the same X and K , compute the valid mathematical convolution output at the upper-left patch by flipping the kernel first.
- Exercise 5.** For a one-dimensional input $x = (x_1, x_2, x_3, x_4)^T$ and kernel $k = (a, b)$, write the valid cross-correlation matrix C_k such that $C_k x$ gives the sliding dot-product output.
- Exercise 6.** Compute the output height and width for a 32×32 input, a 5×5 kernel, padding 2, stride 1, and dilation 1.
- Exercise 7.** Compute the parameter count for a 3×3 convolution with 3 input channels and 16 output channels, including one bias per output channel.
- Exercise 8.** Explain in two or three sentences how parameter sharing differs from a dense layer on a flattened image.
- Exercise 9.** A feature detector responds strongly to a vertical stroke in the center of an image. Explain what translation equivariance predicts if the same stroke moves one pixel to the right, and state why a boundary can break the exact statement.

Exercise 10. Trace the shapes in this pipeline: $28 \times 28 \times 1$ input, 3×3 convolution with 12 output channels and padding 1, ReLU, 2×2 pooling with stride 2, flatten, and a dense layer with 10 outputs.

4.13 Homework: Symbolic Convolution

Required Work

Use SymPy [145] to verify three claims from this chapter: sliding dot products can be written as sparse matrix multiplication, fixed-kernel correlation is linear, and the output-shape formula matches concrete symbolic arrays.

Submission model. Submit the completed starter script, the main hand derivation, the SymPy output needed to verify it, and a short interpretation. External computer-algebra tools may be used only as optional sanity checks.

Task 1. Define symbolic input entries $x_0, \dots, x_4$ and kernel entries k_0, k_1, k_2 .

Task 2. Construct the valid one-dimensional cross-correlation output for the length-five input and length-three kernel.

Task 3. Build the sparse matrix C_k whose product with the input vector gives the same output.

Task 4. Use SymPy to verify that $C_k x$ equals the sliding dot-product output.

Task 5. Define two symbolic input vectors u and v , then verify linearity by simplifying $C_k(\alpha u + \beta v) - \alpha C_k u - \beta C_k v$.

Task 6. Define a symbolic 3×3 image and a symbolic 2×2 kernel.

Task 7. Construct the 2×2 valid cross-correlation output for the symbolic image and kernel.

Task 8. Substitute small integer values into the two-dimensional expression and compare the result with a hand calculation for the upper-left patch.

Task 9. Use the output-shape formula to compute the spatial shape for $h = w = 28$, $k_h = k_w = 3$, $p_h = p_w = 1$, $s_h = s_w = 1$, and $d_h = d_w = 1$.

Task 10. Write one paragraph explaining why the sparse-matrix view supports the claim that a convolutional layer is a structured linear operator before the activation is applied.

Reference Values for Checking

For $x = (x_0, x_1, x_2, x_3, x_4)^T$ and $k = (k_0, k_1, k_2)$, the valid output is

$$\begin{bmatrix} k_0 x_0 + k_1 x_1 + k_2 x_2 \\ k_0 x_1 + k_1 x_2 + k_2 x_3 \\ k_0 x_2 + k_1 x_3 + k_2 x_4 \end{bmatrix}.$$

The corresponding matrix is

$$\begin{bmatrix} k_0 & k_1 & k_2 & 0 & 0 \\ 0 & k_0 & k_1 & k_2 & 0 \\ 0 & 0 & k_0 & k_1 & k_2 \end{bmatrix}.$$

The linearity check should simplify to the zero vector.

For a 3×3 image and 2×2 kernel, the valid output has shape 2×2 . With the integer image and kernel values in the starter file, the numeric 2×2 output should be

$$\begin{bmatrix} 1 & 8 \\ 1 & -2 \end{bmatrix}.$$

The upper-left entry is the hand check

$$1 \cdot 1 + (-1) \cdot 2 + 0 \cdot 0 + 2 \cdot 1 = 1.$$

For $h = w = 28$, $k_h = k_w = 3$, padding 1, stride 1, and dilation 1, the output shape is 28×28 .